

AGREEMENT PROTOCOLS

Agreement Protocols:

- In distributed systems, where sites (or processors) often compete as well as cooperate to achieve a common goal, it is often required that sites reach mutual agreement.
- For example, in distributed database systems, data managers at sites must agree on whether to commit or to abort a transaction .
- Reaching an agreement typically requires that sites have knowledge about the values of other sites. For example, in *distributed commit*, a site should know the outcome of *local commit* at each site.

Agreement Protocols:

- When the system is free from failures, an agreement can easily be reached among the processors (or sites).
- For example, processors can reach an agreement by communicating their values to each other and then by taking a majority vote or a minimum, maximum, mean, etc. of those values.
- However, when the system is prone to failure, this method does not work. This is because faulty processors can send conflicting values to other processors preventing them from reaching an agreement.

Agreement Protocols:

- In the presence of faults, processors must exchange their values with other processors and relay the values received from other processors several times to isolate the effects of faulty processors.
- A processor refines its value as it learns of the values of other processors (This entire process of reaching an agreement is called an *agreement protocol*).
- we study agreement protocols for distributed systems under processor failures. A very general model of faults is assumed.
- For example, a faulty processor may send spurious messages to other processors, may lie, may not respond to received messages correctly, etc. Also, nonfaulty processors do not know which processors are faulty

Agreement Protocols:

- In agreement problems, nonfaulty processors in a distributed system should be able to reach a common agreement, even if certain components in the system are faulty.
- The agreement is achieved through an agreement protocol that involves several rounds of message exchange among the processors.

THE SYSTEM MODEL

- Agreement problems have been studied under the following system model:
 - There are n processors in the system and at most m of the processors can be faulty.
 - The processors can directly communicate with other processors by message passing. Thus, the system is logically fully connected.
 - A receiver processor always knows the identity of the sender processor of the message.
 - The communication medium is reliable (i.e., it delivers all messages without introducing any errors) and only processors are prone to failures.
- For simplicity, we assume that agreement is to be reached between only two values, 0 and 1.

Synchronous vs. Asynchronous Computations

- In a synchronous computation, processes in the system run in lock step manner, where in each step, a process receives messages (sent to it in the previous step), performs a computation, and sends messages to other processes (received in the next step).
- A step of a synchronous computation is also referred to as a *round*.
- In synchronous computation, a process knows all the messages it expects to receive in a round.

Synchronous vs. Asynchronous Computations

- In an asynchronous computation, on the other hand, the computation at processes does not proceed in lock steps. A process can send and receive messages and perform computation at any time.
- **In** this model, synchronous models of computation are assumed.
- the agreement problem is not solvable in an asynchronous system, even for a single processor failure [10].

Model of Processor Failures

- In agreement problems A processor can fail in three modes: *crash fault*, *omission fault*, and *malicious fault* .
- In a crash fault, a processor stops functioning and never resumes operation.
- In an omission fault a processor "omits" to send messages to some processors. (These are the messages that the processor should have sent according to the protocol or algorithm it is executing.)
- For example, a processor is supposed to broadcast a message to all other processors, but it sends the message to only a few processors

Model of Processor Failures

- In a malicious fault, a processor behaves randomly and arbitrarily. For example, a processor may send fictitious messages to other processors to confuse them.
- Malicious faults are also referred to as *Byzantine faults*.
- Since a faulty processor can refuse to send a message, a nonfaulty processor may never receive an expected message from a faulty processor.
- In such a situation, we assume that the nonfaulty processor simply chooses an arbitrary value and acts as if the expected message has been received.
- In synchronous systems, if the duration of each round is known, then this detection is simple-all the expected messages not received by the end of a round were not sent.

Authenticated vs. Non-Authenticated Messages

- To reach an agreement, processors have to exchange their values and relay the received values to other processors several times.
- The capability of faulty processors to distort what they receive from other processors greatly depends upon the type of underlying messages.
- There are two types of messages: *authenticated* and *non-authenticated*.
- In an authenticated message system, a (faulty) processor cannot forge a message or change the contents of a received message (before it relays the message to other processors).
- An authenticated message is also called a *signed* message [14].

Authenticated vs. Non-Authenticated Messages

- In a non-authenticated message system, a (faulty) processor can forge a message and claim to have received it from another processor or change the contents of a received message before it relays the message to other processors.
- A processor has no way of verifying the authenticity of a received message.
- A non-authenticated message is also called an *oral* message
- It is easier to reach agreement in an authenticated message system because faulty processors are capable of doing less damage.

Performance Aspects

- The performance (or the computational complexity) of agreement protocols is generally determined by the following three metrics: *time*, *message traffic*, and *storage overhead*.
- Time refers to the time taken to reach an agreement under a protocol.
- The time is usually expressed as the number of rounds needed to reach an agreement.
- Message traffic is measured by the number of messages exchanged to reach an agreement.
- Sometimes, the message traffic is also measured by the total number of bits exchanged to reach an agreement [5]
- Storage overhead measures the amount of information that needs to be stored at processors during the execution of a protocol

A CLASSIFICATION OF AGREEMENT PROBLEMS

- Next, we discuss three agreement problems for non-authenticated messages under processor failures.
- There are three well known agreement problems in distributed systems: the *Byzantine agreement problem*, the *consensus problem*, and the *interactive consistency problem*
- **In the Byzantine agreement problem**, a single value, which is to be agreed on, is initialized by an arbitrary processor and all nonfaulty processors have to agree on that value.
- **In the consensus problem**, every processor has its own initial value and all nonfaulty processors must agree on a single common value.
- **In the interactive consistency problem**, every processor has its own initial value and all nonfaulty processors must agree on a *set* of common values.

A CLASSIFICATION OF AGREEMENT PROBLEMS

- In all three problems, all nonfaulty processors must reach a common agreement.
- In the Byzantine agreement and the consensus problems, the agreement is about a single value.
- Whereas in the interactive consistency problem, the agreement is about a set of common values.
- In the Byzantine agreement problem, only one processor initializes the initial value. Whereas in the consensus and the interactive consistency problems, every processor has its own initial value.

The three agreement problems

Problem →	Byzantine Agreement	Consensus	Interactive Consistency
Who initiates the value	One processor	All processors	All processors
Final agreement	Single value	Single value	A vector of values

A CLASSIFICATION OF AGREEMENT PROBLEMS

8.3.1 The Byzantine Agreement Problem

In the Byzantine agreement problem, an arbitrarily chosen processor, called the *source processor*, broadcasts its initial value to all other processors. A solution to the Byzantine agreement problem should meet the following two objectives:

Agreement. All nonfaulty processors agree on the same value.

Validity. If the source processor is nonfaulty, then the common agreed upon value by all nonfaulty processors should be the initial value of the source.

A CLASSIFICATION OF AGREEMENT PROBLEMS

8.3.2 The Consensus Problem

In the consensus problem, every processor broadcasts its initial value to all other processors. Initial values of the processors may be different. A protocol for reaching consensus should meet the following conditions:

Agreement All nonfaulty processors agree on the same single value.

Validity If the initial value of every nonfaulty processor is v , then the agreed upon common value by all nonfaulty processors must be v .

Note that if the initial values of nonfaulty processors are different, then all non-faulty processors can agree on *any* common value. Again, we don't care what value faulty processors agree on.

A CLASSIFICATION OF AGREEMENT PROBLEMS

8.3.3 The Interactive Consistency Problem

In the interactive consistency problem, every processor broadcasts its initial value to all other processors. The initial values of the processors may be different. A protocol for the interactive consistency problem should meet the following conditions:

Agreement. All nonfaulty processors agree on the same vector, $(v_1, v_2, \dots, v_n)$.

Validity. If the i th processor is nonfaulty and its initial value is v_i , then the i th value to be agreed on by all nonfaulty processors must be v_i .

Note that if the j th processor is faulty, then all nonfaulty processors can agree on *any* common value for v_j . It is irrelevant what value faulty processors agree on.

SOLUTIONS TO THE BYZANTINE AGREEMENT PROBLEM

- The Byzantine agreement problem was first defined and solved (under processor failures) by Lamport et al.
- in this problem, an arbitrarily chosen processor (called the source processor) broadcasts its initial value to all other processors.
- A protocol for the Byzantine agreement should guarantee that all nonfaulty processors agree on the same value and if the source processor is nonfaulty, then the common agreed upon value by all nonfaulty processors should be the initial value of the source.
- It is obvious that all the processors must exchange the values through messages to reach a consensus.
- Processors send their values to other processors and relay received values to other processors
- During the execution of the protocol, faulty processors may confuse other processors by sending them conflicting values or by relaying to them fictitious values.

Byzantine *generals* problem

- The Byzantine agreement problem is also referred to as the Byzantine *generals* problem. because the problem resembles a situation where a team of generals in an army is trying to reach agreement on an attack plan.
- The generals are located at geographically distant positions and communicate only through messengers.
- Some of the generals are traitors (equivalent to faulty processors) and try to prevent loyal generals from reaching an agreement by deliberately transmitting erroneous information.

The Upper Bound on the Number of Faulty Processors

- In order to reach an agreement on a common value, nonfaulty processors need to be free from the influence of faulty processors.
- If faulty processors dominate in number, they can prevent nonfaulty processors from reaching a consensus.
- Thus, the number of faulty processors should not exceed a certain limit if a consensus is to be reached.
- Pease et al. [16] showed that in a fully connected network, it is impossible to reach a consensus if the number of faulty processors, m , exceeds [lower bound of $(n - 1)/3$].
- Lamport et al. [14] were the first to give a protocol to reach Byzantine agreement that requires $m + 1$ rounds of message exchanges (m is the maximum number of faulty processors).
- Fischer et al. [8] showed that $m + 1$ is the lower bound on the number of rounds of message exchanges to reach a Byzantine agreement in a fully connected network where only processors can fail. However, if authenticated messages are used, this bound is relaxed and a consensus can be reached for any number of faulty processors.

An Impossibility Result

- Byzantine agreement cannot be reached among three processors, where one processor is faulty.
- Consider a system with three processors, p_0 , p_1 , and p_2 . For simplicity, we assume that there are only two values, 0 and 1, on which processors agree and processor p_0 initiates the initial value. There are two possibilities:
 - (1), p_0 is not faulty or
 - (2) p_0 is faulty.

An Impossibility Result

- **Case 1: P_0 is not faulty.** Assume p_2 is faulty. Suppose that p_0 broadcasts an initial value of 1 to both P_1 and p_2 . Processor p_2 acts maliciously and communicates a value of 0 to processor P_1 . Thus, P_1 receives conflicting values from P_0 and P_2 . (This scenario is shown in Fig. A faulty processor is depicted by an oval and a nonfaulty processor is denoted by a circle.) However, since P_0 is nonfaulty, processor P_1 must accept 1 as the agreed upon value.

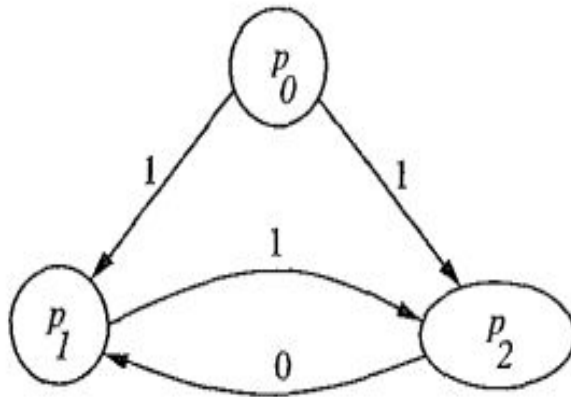


FIGURE 8.1
Processor p_0 is Non-Faulty.

An Impossibility Result

- **Case II: p_0 is faulty.** Suppose that processor p_0 sends an initial value of 1 to p_1 and 0 to p_2 . Processor p_2 will communicate the value 0 to p_1 . (This scenario is shown in Fig. 8.2). As far as p_1 is concerned, this case will look identical to Case I. So any agreement protocol which works for three processors cannot distinguish between the two cases and must force p_1 to accept 1 as the agreed upon value whenever p_1 is faced with such situations. Using a similar argument, we can show that if p_2 receives an initial value of 0 from p_0 , then it must take 0 as the agreed upon value, even if p_1 communicates a value of 1. However, if this is followed in Case II, p_1 will agree on a value of 1 and p_2 will agree on a value of 0, which will violate condition of the solution.
- Therefore, no solution exists for the Byzantine agreement problem for three processors, which can work under single processor failure.

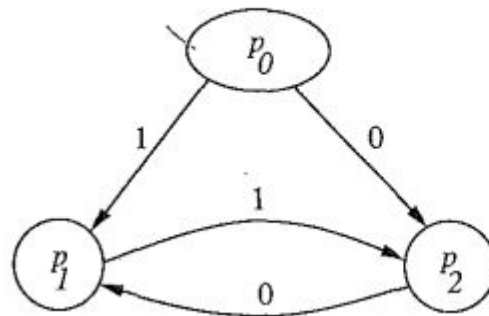


FIGURE 8.2
Processor p_0 is Faulty.

Lamport-Shostak-Pease Algorithm

- Lamport et al.'s algorithm [14], referred to as the Oral Message algorithm OM(m), $m > 0$, solves the Byzantine agreement problem for $3m + 1$ or more processors in the presence of at most m faulty processors. Let n denote the total number of processors (clearly, $n \geq 3m + 1$). The algorithm is recursively defined as follows:

Algorithm OM(0).

1. The source processor sends its value to every processor.
2. Each processor uses the value it receives from the source. (If it receives no value, then it uses a default value of 0.)

Algorithm OM(m), $m > 0$.

1. The source processor sends its value to every processor.
2. For each i , let v_i be the value processor i receives from the source. (If it receives no value, then it uses a default value of 0.). Processor i acts as the new source and initiates **Algorithm OM($m-1$)** wherein it sends the value v_i to each of the $n - 2$ other processors.
3. For each i and each j ($j \neq i$), let v_j be the value processor i received from processor j in Step 2. using Algorithm OM($m-1$). (If it receives no value, then it uses a default value of 0.). Processor i uses the value $\text{majority}(v_1, v_2, \dots, v_{n-1})$.

Lamport-Shostak-Pease Algorithm

- Consider a system with four processors, p_0 , p_1 , p_2 , and p_3 . For simplicity, we assume that there are only two values 0 and 1, furthermore, we assure that processor p_0 initiates the initial value and that processor p_2 is faulty.
- To initiate the agreement, processor p_0 executes algorithm OM(1) wherein it sends its value 1 to all other processors (Fig. 8.3). At Step 2 of the algorithm OM(1), after having received the value 1 from the source processor p_0 , processors p_1 , p_2 , and p_3 execute the algorithm OM(0). These executions are shown in Fig. 8.4. Processors p_1 and p_3 are nonfaulty and send value 1 to processors $\{p_2, p_3\}$ and $\{p_1, p_2\}$, respectively. Faulty processor p_2 sends value 1 to p_1 and a value 0 to p_3 (see Fig. 8.4)

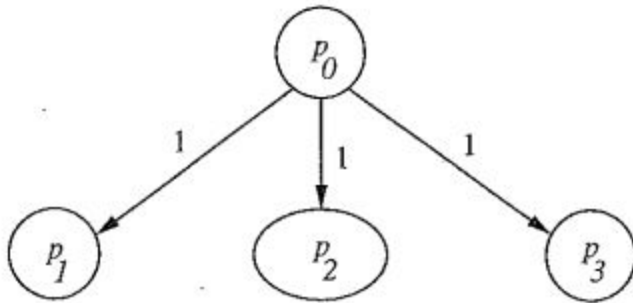


FIGURE 8.3
Processor p_0 executes the algorithm OM(1)

Lamport-Shostak-Pease Algorithm

- After having received all the messages, processors p_1 , p_2 , and p_3 execute Step 3 of the algorithm OM(1) to decide on the majority value. Processor p_1 has received values (1, 1, 1), whose majority value is 1, processor p_2 has received values (1, 1, 1), whose majority value is 1, and processor p_3 has received values (1, 1, 0), whose majority value is 1. Thus, both conditions of the Byzantine agreement are satisfied.

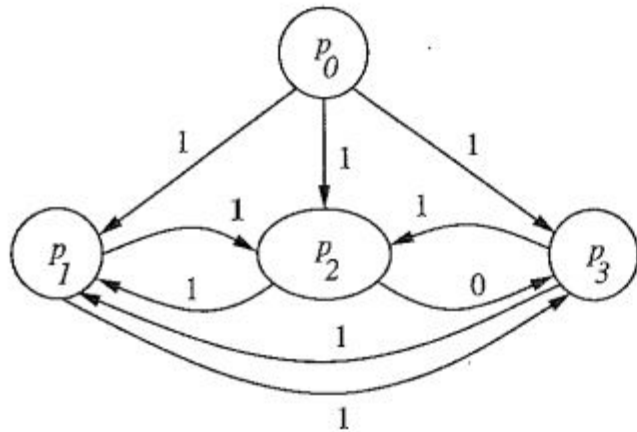


FIGURE 8.4

Processors p_1 , p_2 , and p_3 execute the algorithm OM(0)

Lamport-Shostak-Pease Algorithm

- Figure 8.5 shows a situation where processor p_0 is faulty and sends conflicting values to the other three processors. These three processors, under Step 1 of OM(O), send the received values to the other two processors.
- After having received all the messages, processors p_1 , p_2 , and p_3 execute Step 3 of the algorithm OM(1) to decide on the majority value. Note that all three processors have received values (1, 0, 1), whose majority value is 1. Thus, all three nonfaulty processors agree on the same value and the required conditions of the Byzantine agreement are satisfied

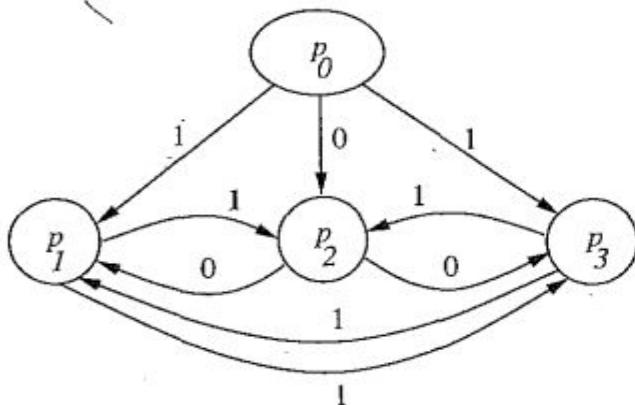


FIGURE 8.5

Processors p_0 , p_1 , p_2 , and p_3 execute OM(1) and OM(0)